

1 FUNDAMENTOS E AMBIENTE DE DESENVOLVIMENTO MOBILE.....	3
1.2 Conceito de aplicações móveis.....	3
1.3 Diferenças entre aplicações nativas, híbridas e multiplataforma.....	4
1.4 Arquitetura de sistemas mobile.....	5
1.4.1 O padrão MVC (model-view-controller).....	5
1.5 Contexto de uso e experiência do usuário (UX) no mobile.....	6
1.5.1 Primeiro app Flutter.....	7
2 AMBIENTES DE DESENVOLVIMENTO E FERRAMENTAS.....	9
2.1 Ferramentas utilizadas no desenvolvimento Mobile.....	9
2.4 Arquitetura e estrutura de diretórios em projetos mobile.....	10
2.5 Boas práticas e qualidade de software mobile.....	13
2.5.1 Otimização de performance e assincronismo.....	13
3.1 O papel do Android Studio no desenvolvimento mobile profissional.....	15
3.2 Estrutura de construção de interfaces XML.....	16
3.3 Componentes fundamentais de interface.....	17
3.4 Organização de layouts e hierarquia visual.....	18
3.5 Responsividade e adaptação em layouts Android.....	19
3.5.1 Tela de autenticação funcional.....	21
3.6 Boas práticas e acessibilidade na construção de interfaces.....	21
4 DESENVOLVIMENTO MULTIPLATAFORMA COM FLUTTER.....	22
4.1 Introdução ao Flutter e a engenharia multiplataforma.....	22
4.2 Estrutura arquitetural de um aplicativo Flutter.....	23
4.3 Construção de interfaces baseada em widgets.....	23
4.4 Layouts e organização de componentes.....	25
4.5 Responsividade e adaptação de interfaces em Flutter.....	27
4.6 Gerenciamento de estado e reatividade.....	28
4.6.1 Abordagens de gerenciamento de estado moderno.....	30
4.7 Garantia de qualidade e testes de interface.....	32

4.8 Boas práticas e governança de código no Flutter.....	33
5 INTEGRAÇÃO DE DADOS E FUNCIONALIDADES AVANÇADAS.....	33
5.1 Consumo de dados e arquitetura de integração com APIs.....	33
5.2 Arquitetura cliente-servidor e o papel estratégico das APIs.....	34
5.3 Comunicação assíncrona e protocolo HTTP.....	34
5.4 Manipulação e serialização de dados (JSON).....	36
5.5 Tratamento de exceções e resiliência em sistemas móveis.....	37
5.6 Exemplo prático integrado: consumo de endpoints REST.....	39
6.1 Estratégias de persistência de dados local.....	40
6.2 Persistência estruturada com SQLite.....	41
6.3 Integração com recursos de hardware e sensores.....	42
6.4 Arquitetura offline-first e governança de permissões.....	44
6.5 Sincronização de dados e testes de integração.....	46
7 ENGENHARIA E ARQUITETURA DE APLICAÇÕES MOBILE.....	48
7.1 Estrutura de aplicações mobile modernas e padronização arquitetural.....	48
7.2 Engenharia de navegação e fluxo de estados.....	49
7.3 Implementação de sistemas de autenticação e gestão de sessão.....	50
7.4 Engenharia de validação e integridade de dados.....	52
7.5 Arquitetura de software: organização e separação de responsabilidades.....	55
7.6 Engenharia de gerenciamento de estado.....	56
7.7 Garantia de qualidade: testes e estabilidade da aplicação.....	58
8 SEGURANÇA, ENGENHARIA DE QUALIDADE E PUBLICAÇÃO.....	59
8.1 Autenticação de usuários e gestão segura de sessão.....	59
8.2 Engenharia de segurança e proteção de ativos digitais.....	60
8.3 Engenharia de qualidade: estratégias de testes automatizados.....	62
8.4 Testes multidispositivo, fragmentação e responsividade.....	63
8.5 Ciclo de lançamento: otimização, build e versionamento.....	64
8.6 Engenharia de lançamento e ecossistemas de distribuição.....	66
8.7 Síntese de engenharia: orquestração de uma aplicação completa.....	66

Unidade I

1 FUNDAMENTOS E AMBIENTE DE DESENVOLVIMENTO MOBILE

O desenvolvimento mobile consiste na concepção e construção de aplicações voltadas para dispositivos portáteis, como smartphones, tablets e wearables (dispositivos vestíveis).

1.2 Conceito de aplicações móveis

Ao contrário do desenvolvimento para desktop, o ecossistema mobile impõe desafios singulares que exigem uma abordagem multidisciplinar, unindo engenharia de software de alta performance e um design focado estritamente no contexto do usuário.

O projeto de um aplicativo é condicionado por restrições de hardware e variáveis de ambiente que definem o sucesso ou o fracasso da solução. As principais características deste cenário incluem:

- Restrições de hardware: diferente dos computadores de mesa, dispositivos móveis possuem limitações de processamento, memória RAM e, principalmente, autonomia de bateria. O desenvolvedor deve prever o consumo energético para evitar a rejeição do software pelo usuário.
- Fragmentação de interface: existe a necessidade de adaptação a uma vasta gama de resoluções, densidades de pixels e proporções de tela (aspect ratios).
- Interatividade tátil: a interface deve ser projetada para gestos e toques, considerando a precisão do dedo humano em substituição aos periféricos de ponta (como o mouse).
- Conectividade intermitente: o sistema deve ser resiliente a oscilações de rede (4G/5G/Wi-Fi), prevendo o funcionamento offline ou com sincronização assíncrona.
- Contextos de uso dinâmicos: o usuário interage com o dispositivo em ambientes com alta carga de distração (movimento, ruído, luz solar direta), o que exige foco total na tarefa principal.

O design de interfaces móveis deve priorizar a simplicidade estética e a clareza visual, com foco fundamental na redução da carga cognitiva. Isso significa minimizar

o esforço mental necessário para processar informações, garantindo que o sistema seja intuitivo e reduza a dependência da memória de curto prazo do usuário.

1.3 Diferenças entre aplicações nativas, híbridas e multiplataforma

As aplicações nativas são construídas especificamente para um sistema operacional (OS), utilizando as linguagens e ferramentas oficiais fornecidas pela Google (Kotlin ou Java para Android) ou pela Apple (Swift para iOS).

Por operarem sem camadas intermediárias (bridges), estas aplicações oferecem o máximo desempenho e acesso irrestrito às APIs de hardware (sensores, biometria, câmera). No entanto, essa abordagem exige a manutenção de dois códigos-fonte distintos, o que eleva substancialmente o custo de produção e demanda equipes especializadas para cada ecossistema.

Surgindo como uma alternativa de baixo custo, as aplicações híbridas utilizam tecnologias web tradicionais (HTML5, CSS3 e JavaScript) encapsuladas em um container nativo. Elas operam essencialmente através de uma WebView (um navegador interno ao app). Embora permitam o reaproveitamento quase total do código entre plataformas, elas frequentemente sofrem com limitações de fluidez visual e latência na execução de tarefas complexas, uma vez que dependem do motor de renderização do navegador do dispositivo.

As aplicações multiplataforma representam a evolução do desenvolvimento mobile, permitindo a criação de apps para Android e iOS a partir de um único código-fonte. Diferente das híbridas, tecnologias modernas como o Flutter conseguem entregar um desempenho próximo ao nativo.

- Flutter (Google): utiliza a linguagem Dart e renderiza seus próprios componentes gráficos através do motor Impeller/Skia, garantindo que a interface seja idêntica em qualquer plataforma.
- React Native (Meta): utiliza JavaScript a fim de mapear componentes para widgets nativos.
- .NET MAUI (Microsoft): permite o uso de C# para desenvolvimento multiplataforma, trata-se de uma evolução do Xamarin.

A escolha da arquitetura não é absoluta. Aplicações que exigem processamento gráfico intensivo (como jogos) tendem ao nativo; MVPs (produtos mínimos viáveis) com prazos curtíssimos podem optar pelo híbrido; enquanto aplicações corporativas

modernas, que buscam alta fidelidade visual e agilidade, encontram na multiplataforma (Flutter) o equilíbrio ideal entre desempenho e produtividade.

1.4 Arquitetura de sistemas mobile

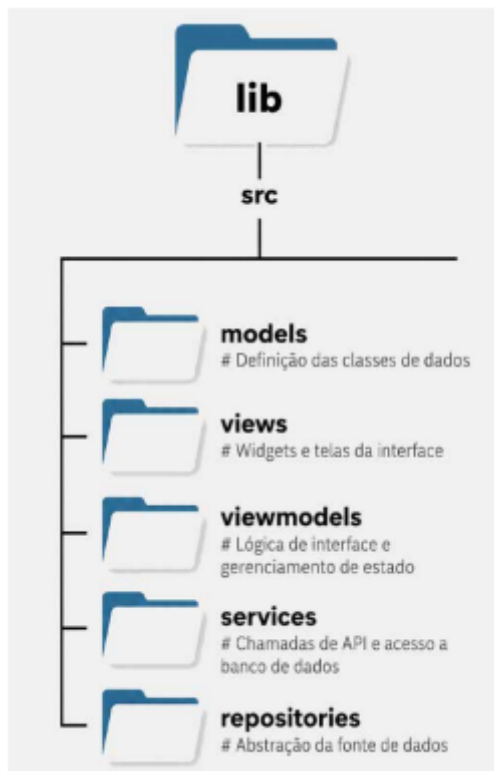
A arquitetura de software é a estrutura organizacional que define como os componentes de uma aplicação se comunicam, como os dados são processados e como as responsabilidades são segregadas. Uma arquitetura robusta visa garantir:

- Desacoplamento: redução da dependência direta entre componentes, facilitando a troca de bibliotecas ou serviços.
- Testabilidade: possibilidade de realizar testes unitários na lógica de negócio sem depender da interface gráfica.
- Escalabilidade: capacidade do sistema crescer em funcionalidades sem se tornar excessivamente complexo ou instável.

1.4.1 O padrão MVC (model-view-controller)

- Model (modelo): representa o domínio dos dados e as regras de negócio. É responsável por buscar, validar e persistir informações (seja em bancos de dados locais como SQLite ou via APIs REST).
- View (visão): é a camada de apresentação. No Flutter, corresponde à árvore de widgets que o usuário visualiza e interage.
- Controller (controlador): atua como intermediário. Ele interpreta as ações do usuário na view, aciona o model e devolve a resposta para a interface.
- MVVM (model-view-viewmodel): essencial no Flutter. A viewmodel não comanda a view diretamente (como o controller faz); em vez disso, ela expõe um estado que a view observa. Quando o dado muda, a tela se reconstrói automaticamente.
- Clean architecture (arquitetura limpa): proposta por Martin (2019), divide o sistema em círculos concêntricos, onde as dependências apontam apenas para dentro. Isso torna a lógica de negócio (casos de uso) totalmente independente de frameworks externos ou bancos de dados.

Para implementar essas arquiteturas de forma profissional, não basta organizar pastas; é necessário utilizar ferramentas que controlem o fluxo de dados. Vemos isso no ecossistema Flutter, onde o gerenciamento de estado é o coração da arquitetura. Bibliotecas como Provider, Riverpod e BLoC (Business Logic Component) são utilizadas para garantir que os dados fluam de forma unidirecional e previsível, evitando comportamentos inesperados na interface.



1.5 Contexto de uso e experiência do usuário (UX) no mobile

O contexto de uso é o fator determinante que diferencia as aplicações móveis dos sistemas tradicionais de desktop. Enquanto no computador de mesa o usuário opera em um ambiente controlado e estável, o cenário mobile é caracterizado pela intermitência e pela mobilidade.

- Mobilidade constante: o uso ocorre em trânsito (caminhando, em transportes públicos), o que exige interfaces que possam ser operadas com apenas uma das mãos.
- Sessões fragmentadas: o usuário utiliza o aplicativo por períodos curtos, focados em resolver uma necessidade imediata (por exemplo, conferir um horário ou confirmar um pagamento).
- Atenção dividida: a interação é frequentemente interrompida por estímulos externos (ruído, tráfego, notificações), exigindo que a interface minimize a carga cognitiva.
- Navegação por micromomentos: prioriza-se a rapidez na obtenção da resposta sobre a profundidade da informação.

1.5.1 Primeiro app Flutter

A compreensão da anatomia de uma aplicação Flutter é fundamental para dominar o fluxo de renderização e o gerenciamento de estados. O framework utiliza a linguagem Dart para construir interfaces de forma declarativa, onde a interface reflete o estado atual do sistema.

```
// O ponto de entrada obrigatório para qualquer aplicação Dart
void main() {
  runApp(const MeuPrimeiroApp());
}

class MeuPrimeiroApp extends StatelessWidget {
  const MeuPrimeiroApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'ADS - Desenvolvimento Mobile',
      theme: ThemeData(primarySwatch: Colors.blue),
      home: const TelaPrincipal(),
    );
  }
}
```

Unidade I

```
}

class TelaPrincipal extends StatelessWidget {
  const TelaPrincipal({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Primeiro Aplicativo Flutter'),
      ),
      body: const Center(
        child: Text(
          'Olá, Mundo Mobile!',
          style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
        ),
      ),
    );
  }
}
```

- `main()`: define o ponto de entrada da execução. Em aplicações Flutter, sua única responsabilidade inicial é invocar a função `runApp()`, que vincula o widget raiz à árvore de renderização.
- `MaterialApp`: este widget atua como o wrapper de conveniência que introduz o material design. Ele configura o navegador, o sistema de temas e as diretrizes visuais padrão do Google.
- `Scaffold`: implementa a estrutura visual básica do material design. Ele fornece espaços predefinidos para elementos comuns de UI, como a `AppBar` (barra superior), `Drawer` (menu lateral) e o `FloatingActionButton`.
- `StatelessWidget`: representa um widget que não requer mutabilidade de estado. Uma vez construído, ele não se altera em resposta a eventos internos, o que otimiza o desempenho da renderização inicial.
- `Center` e `Text`: demonstram a natureza composicional do Flutter. O `center` é um widget de layout que centraliza seu filho (`child`) no espaço disponível, enquanto o `text` é o componente fundamental para renderização de cadeias de caracteres com estilos específicos.

O Flutter utiliza dois modos de compilação: JIT (Just-In-Time) durante o desenvolvimento, o que permite o recurso de hot reload (atualização quase instantânea do código na tela); e AOT (Ahead-Of-Time) para a versão final de produção. A compilação AOT transforma o código Dart diretamente em código de máquina nativo (ARM ou x64), garantindo o alto desempenho exigido para aplicações profissionais.

2 AMBIENTES DE DESENVOLVIMENTO E FERRAMENTAS

2.1 Ferramentas utilizadas no desenvolvimento Mobile

Android Studio

O Android Studio é a IDE oficial para a plataforma Android e é baseada no ecossistema IntelliJ IDEA. Para o desenvolvedor de sistemas, ele não é apenas um editor de texto, mas um centro de operações que integra:

- Editor inteligente: suporte avançado para refatoração e autocompletar para linguagens como Kotlin, Java e

Dart. • Android SDK Manager: ferramenta crítica para gerenciar versões da API e ferramentas de build. • Android Emulator: criação de Dispositivos Virtuais Android (AVDs) com suporte à aceleração de hardware. • Profiler de desempenho: monitoramento em tempo real do consumo de CPU, memória e rede da aplicação.

Xcode

O Xcode representa o ambiente oficial da Apple para o desenvolvimento no ecossistema iOS, iPadOS e macOS. Diferente de outras ferramentas, o Xcode é exclusivo para o sistema operacional macOS.

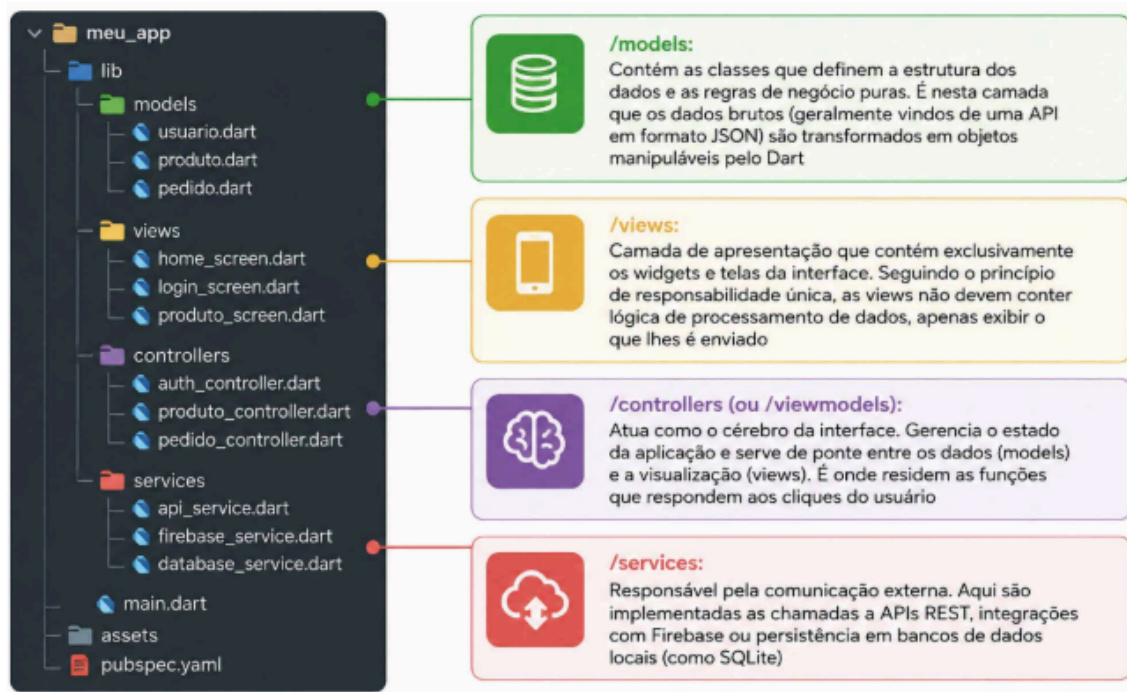
- Flutter: desenvolvido pelo Google, este framework utiliza a linguagem Dart. Sua principal distinção técnica é a renderização própria, pois o Flutter não converte seus componentes para widgets nativos do sistema; ele os desenha pixel a pixel através de um motor gráfico de alto desempenho (Skia/Impeller). Isso garante que a interface seja idêntica em Android e iOS.
- .NET MAUI: sucessor do Xamarin, o framework da Microsoft utiliza C# e XAML. Ao contrário do Flutter, o MAUI mapeia seus controles para os componentes nativos de cada plataforma. Essa abordagem é preferida em ambientes corporativos que já utilizam o ecossistema .NET e buscam uma aparência 100% nativa com o comportamento padrão de cada sistema operacional.

2.4 Arquitetura e estrutura de diretórios em projetos mobile

No ecossistema Flutter, embora o framework não imponha uma estrutura rígida, o mercado adota padrões baseados em clean architecture e MVVM.

- /models: contém as classes que definem a estrutura dos dados e as regras de negócio puras. É nesta camada que os dados brutos (geralmente vindos de uma API em formato JSON) são transformados em objetos manipuláveis pelo Dart.
- /views: camada de apresentação que contém exclusivamente os widgets e as telas da interface. Seguindo o princípio de responsabilidade única, as views não devem conter lógica de processamento de dados, apenas exibir o que lhes é enviado.
- /controllers (ou /viewmodels): atua como o cérebro da interface. Gerencia o estado da aplicação e serve de ponte entre os dados (models) e a visualização (views). É onde residem as funções que respondem aos cliques do usuário.

- /services: responsável pela comunicação externa. Aqui são implementadas as chamadas APIs REST, integrações com Firebase ou persistência em bancos de dados locais (como SQLite).



Comandos para criar e rodar aplicação:

#Execute os comandos abaixo:

```
flutter create teste_ambiente
```

```
cd teste_ambiente
```

```
flutter run
```

Construção de uma interface de autenticação

```
import 'package:flutter/material.dart';

class TelaLogin extends StatelessWidget {
  const TelaLogin({super.key});
  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.all(20.0),
      child: Column(
        mainAxisAlignment: MainAxisAlignment.center,
        children: [
          const Text(
            'Acesso ao Sistema',
            style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
          ),
        ],
      ),
    );
  }
}
```

```

),
const SizedBox(height: 20),
// Campo de entrada para o Usuário
const TextField(
  decoration: InputDecoration(
    labelText: 'Usuário',
    border: OutlineInputBorder(),
    prefixIcon: Icon(Icons.person),
  ),
),
const SizedBox(height: 15),
// Campo de entrada para a Senha com proteção de caracteres
const TextField(
  obscureText: true,
  decoration: InputDecoration(
    labelText: 'Senha',
    border: OutlineInputBorder(),
    prefixIcon: Icon(Icons.lock),
  ),
),
const SizedBox(height: 20),
ElevatedButton(
  onPressed: () {},
  child: const Text('Entrar'),
),
],
),
);
}
}

```

Análise técnica dos widgets de formulário

- **TextField**: o componente primário para entrada de texto. Ele gerencia o foco, a edição e a interação com o teclado virtual do dispositivo.
- **InputDecoration**: classe responsável pela estética do campo. O atributo `labelText` fornece o rótulo flutuante, essencial para manter o contexto sem ocupar espaço excessivo (padrão Material Design).
- **obscureText: true**: propriedade crítica de segurança que substitui os caracteres digitados por símbolos (como asteriscos ou pontos), impedindo a leitura da senha por terceiros.
- **Column**: widget de layout que organiza os elementos

verticalmente. Em ADS, o uso de column deve ser acompanhado de estratégias de responsividade para evitar o estouro de pixels em telas pequenas.

2.5 Boas práticas e qualidade de software mobile

Com isso, a premissa fundamental do design mobile é a economia de atenção. Segundo Krug (2014), uma interface eficaz é aquela que não obriga o usuário a pensar sobre “como” navegar, permitindo que ele foque apenas no “quê” deseja realizar. Assim, o desenvolvimento mobile deve se atentar a:

- Hierarquia visual: elementos mais importantes (como botões de ação) devem ter peso visual superior.
- Espaçamento (white space): o uso estratégico de espaços vazios evita a fadiga ocular e previne toques acidentais em elementos próximos.

2.5.1 Otimização de performance e assincronismo

A latência é o principal motivo de abandono de aplicativos, a gestão de performance é tratada através da programação assíncrona. No Flutter, utilizamos o padrão Future e async/await para garantir que tarefas pesadas (como requisições de rede) não bloqueiem a Thread principal da interface (UI Thread).

```
// Exemplo de busca de dados assíncrona para evitar travamento da UI
Future<void> buscarDadosServidor() async {
  try {
    // Simula uma requisição de rede
    await Future.delayed(const Duration(seconds: 2));
    print("Dados carregados com sucesso");
  } catch (e) {
    print("Erro na requisição: $e");
  }
}
```

Uma boa prática no Flutter é minimizar os rebuilds desnecessários. Ao utilizar widgets constantes (const) e gerenciar o estado de forma granular (por exemplo,

com Provider ou BLoC), o desenvolvedor garante que apenas a parte alterada da tela seja redesenhada, economizando bateria e processamento do dispositivo.

Segurança e proteção de dados

- Comunicação segura: toda troca de dados deve utilizar o protocolo HTTPS com certificados válidos.
- Sanitização de entradas: validar todos os campos de texto para evitar ataques de injeção.
- Persistência segura: nunca armazenar senhas ou tokens em texto claro no dispositivo. Utilize bibliotecas como `flutter_secure_storage` que utilizam o KeyStore (Android) ou Keychain (iOS).

Estratégias de teste e validação

- Testes unitários: validam funções e classes isoladamente.
- Testes de widget: verificam se a interface e os componentes visuais respondem corretamente aos estímulos.
- Testes de integração: testam o fluxo completo da aplicação, desde a interface até a resposta do banco de dados ou API.

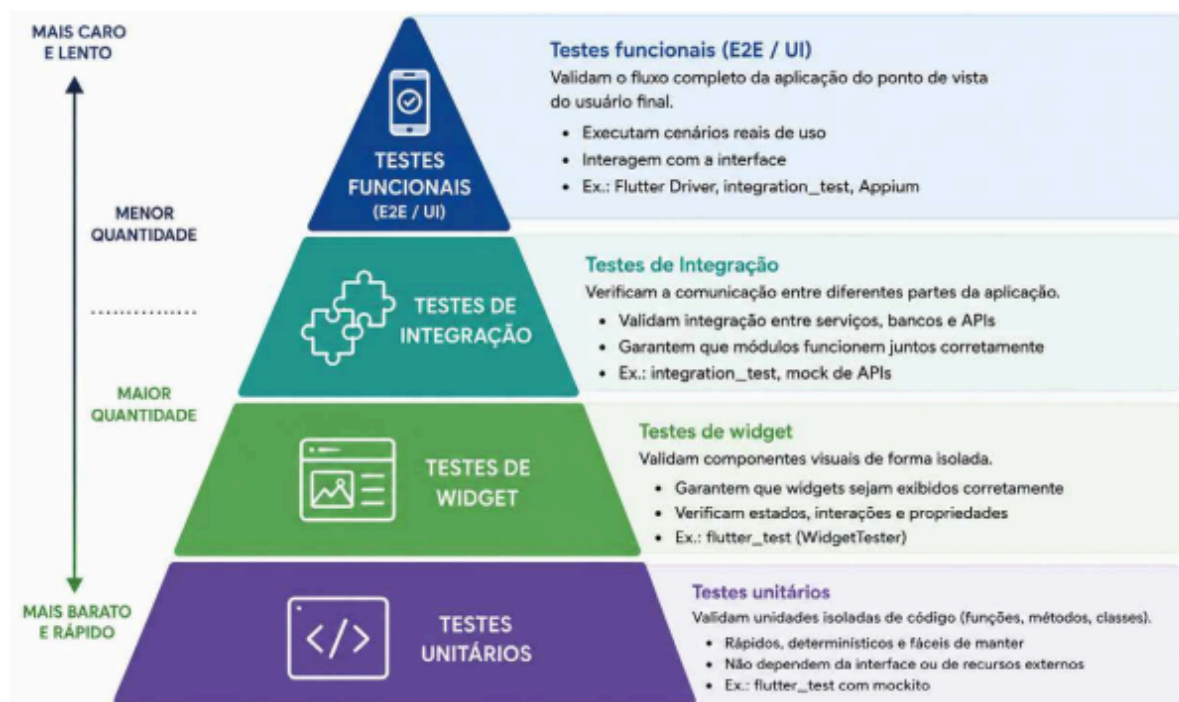


Figura 11 – Pirâmide de testes aplicada ao desenvolvimento mobile: do unitário ao funcional
(imagem produzida com a ferramenta de IA generativa ChatGPT)

Unidade II

3 DESENVOLVIMENTO DE INTERFACES E APLICAÇÕES MULTIPLATAFORMA

A estruturação de interfaces no ecossistema Android pressupõe o entendimento da arquitetura de visualizações (view system). Nesse sentido, o Android Studio atua como o facilitador técnico que permite ao desenvolvedor transpor modelos conceituais de design para estruturas de código declarativo, garantindo que a interface seja não apenas visualmente coerente, mas também eficiente em termos de consumo de recursos computacionais.

3.1 O papel do Android Studio no desenvolvimento mobile profissional

O Android Studio consolidou-se como a IDE (Integrated Development Environment) padrão para o desenvolvimento de aplicações voltadas ao ecossistema Android. Construído sobre a plataforma IntelliJ IDEA, ele provê um ambiente de desenvolvimento altamente especializado que abrange desde o design da interface até a análise de telemetria e performance da aplicação em tempo de execução.

- Editor de código com análise estática: fornece sugestões de código baseadas em padrões de projeto modernos e identifica vulnerabilidades de segurança ou gargalos de performance durante a escrita.
- Preview de interface em tempo real (layout editor): permite a renderização instantânea de layouts XML ou declarativos, facilitando o ajuste de hierarquias visuais sem a necessidade de compilação constante.
- Emulador android integrado: suporta a virtualização de hardware para testes de sensores, diferentes versões do sistema operacional e condições de rede variadas.
- Gerenciamento de dependências (gradle): automatiza o processo de compilação (build) e a integração de bibliotecas externas, garantindo a reprodutibilidade do projeto.
- Ferramentas de análise de desempenho (profilers): essenciais para o diagnóstico de uso de memória, ciclos de CPU e consumo

energético, alinhando-se aos requisitos de eficiência exigidos pelo mercado. • Integração com controle de versão e CI/CD: facilita o trabalho em equipe através do Git e permite a automação de testes e entrega contínua.

3.2 Estrutura de construção de interfaces XML

No desenvolvimento Android nativo, a definição da camada de apresentação (UI) é realizada predominantemente através de arquivos XML (Extensible Markup Language). O XML atua como uma linguagem de marcação declarativa, permitindo que o desenvolvedor descreva a hierarquia e as propriedades dos elementos visuais sem a necessidade de instanciá-los manualmente via código imperativo. Ao definir o layout em um arquivo separado, a aplicação adere ao princípio de separação de responsabilidades, essa dissociação entre a interface e a lógica de controle é fundamental para reduzir o acoplamento, facilitando a manutenção evolutiva e permitindo que designers e desenvolvedores trabalhem em camadas distintas do projeto de forma paralela.

Embora o XML continue sendo amplamente utilizado no desenvolvimento Android nativo, as versões mais recentes do Android também oferecem o Jetpack Compose, uma abordagem moderna para construção de interfaces utilizando Kotlin, baseada em programação declarativa. O estudo de ambas as tecnologias é importante para compreender a evolução do desenvolvimento Android e as tendências adotadas pela indústria.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

DESENVOLVIMENTO MOBILE

```
    android:orientation="vertical"
    android:padding="16dp">
    <TextView
        android:id="@+id/tv_titulo_login"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Acesso ao Sistema"
        android:textSize="24sp"
        android:textStyle="bold"
        android:textColor="@color/black"/>
</LinearLayout>
```

3.3 Componentes fundamentais de interface

Os componentes visuais, fundamentados na classe base View do Android, representam as unidades atômicas utilizadas na composição de interfaces. A escolha estratégica desses elementos não deve ser apenas estética, mas pautada em princípios de usabilidade e acessibilidade.

- **TextView:** utilizado para a renderização de strings de texto na interface. Deve-se observar o uso de unidades sp para garantir a legibilidade e o suporte a ajustes de acessibilidade do sistema operacional.
- **EditText:** especialização do TextView que permite a entrada e edição de dados pelo usuário. É fundamental configurar o inputType (como textPassword ou number) para otimizar a experiência e garantir a segurança básica na entrada de dados.
- **Button:** componente de interação destinado à execução de ações lógicas. Deve fornecer feedback visual claro (como estados de pressionado ou desabilitado) para manter a fluidez da navegação.
- **ImageView:** destinado à exibição de recursos gráficos (vetoriais ou bitmaps). Recomenda-se o uso de descrições de conteúdo (contentDescription) para garantir que leitores de tela possam interpretar a função da imagem.

- **LinearLayout**: um ViewGroup que organiza seus elementos filhos de forma linear (horizontal ou vertical). Embora simples, o uso excessivo de LinearLayouts aninhados pode impactar negativamente a performance de renderização da tela.
- **ConstraintLayout**: o mecanismo de layout mais avançado e recomendado para interfaces modernas. Ele utiliza um sistema de restrições flexíveis que permite criar hierarquias visuais complexas e responsivas sem a necessidade de aninhamentos profundos, otimizando o desempenho da CPU.
- **ScrollView**: um container que permite a rolagem vertical do seu conteúdo. É essencial em dispositivos móveis, onde o espaço de tela é limitado, garantindo que o usuário acesse informações que extrapolam as dimensões físicas do visor.

3.4 Organização de layouts e hierarquia visual

A organização dos elementos na interface não deve ser meramente estética, mas fundamentada em princípios de arquitetura de informação e hierarquia visual. Uma estrutura lógica permite que o sistema nervoso central do usuário processe as informações com maior eficiência, priorizando elementos críticos e organizando a leitura de forma intuitiva.

- **Estabelecer contraste e escala**: utilizar variações de tamanho, cor e peso (por exemplo, títulos em negrito versus textos de apoio) para guiar o olhar do usuário.
- **Agrupar elementos correlatos**: aplicar o princípio da proximidade da Gestalt para indicar que determinados componentes (como campos de input e seus respectivos rótulos) pertencem a uma mesma funcionalidade.
- **Otimizar a árvore de visualizações (view tree)**: manter uma hierarquia de layouts simplificada para acelerar o processo de renderização e reduzir o esforço de processamento do dispositivo.
- **Garantir a escaneabilidade**: facilitar a leitura rápida através de espaçamentos (padding e margins) consistentes, permitindo que o usuário localize informações sem a necessidade de leitura detalhada de cada item.

```

<ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp">
    <TextView
        android:id="@+id/tv_welcome"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Bem-vindo"
        android:textSize="32sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent" />
    <TextView
        android:id="@+id/tv_subtitle"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Faça login para continuar"
        android:textSize="16sp"
        app:layout_constraintTop_toBottomOf="@id/tv_welcome"
        app:layout_constraintStart_toStartOf="parent" />
</ConstraintLayout>

```

3.5 Responsividade e adaptação em layouts Android

Aplicações profissionais devem operar com integridade visual e funcional em uma ampla gama de variáveis, como: dimensões físicas, resoluções de tela, densidades de pixels e orientações (portrait e landscape).

- dp (density-independent pixels): garante que componentes mantenham dimensões físicas consistentes em telas com diferentes densidades.
- sp (scale-independent pixels): unidade mandatória para textos, pois respeita as preferências de acessibilidade do usuário no sistema operacional. A fluidez da interface é controlada primariamente por dois atributos de dimensionamento do Android:
- wrap_content: o componente assume apenas a dimensão necessária para comportar seu conteúdo interno.
- match_parent: o componente expande-se para preencher todo o espaço delimitado pelo seu elemento pai (container).

O ConstraintLayout consolidou-se como o motor de layout padrão para interfaces modernas. Ele permite o posicionamento relativo de elementos através de restrições gráficas, o que resulta em:

- Alta performance: reduz o aninhamento de visualizações (flat view hierarchy), otimizando o tempo de renderização da CPU.
- Flexibilidade: facilita a criação de alinhamentos complexos e proporções dinâmicas que se ajustam automaticamente a tablets ou dispositivos dobráveis.

3.5.1 Tela de autenticação funcional

```
<?xml version="1.0" encoding="utf-8"?>
<ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="24dp">
    <EditText
        android:id="@+id/et_usuario"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="E-mail ou Usuário"
        android:inputType="textEmailAddress"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="32dp"/>
    <EditText
        android:id="@+id/et_senha"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:hint="Senha"
        android:inputType="textPassword"
        app:layout_constraintTop_toBottomOf="@id/et_usuario"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        android:layout_marginTop="16dp"/>
```



```
<Button
    android:id="@+id/btn_entrar"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:text="Acessar Sistema"
    android:layout_marginTop="24dp"
    app:layout_constraintTop_toBottomOf="@id/et_senha"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintEnd_toEndOf="parent"/>
</ConstraintLayout>
```

3.6 Boas práticas e acessibilidade na construção de interfaces

- Redução da carga cognitiva: evitar a saturação de elementos na tela, priorizando o espaço em branco (white space) para guiar o foco do usuário para as ações principais.
- Consistência sistêmica: utilizar padrões de design estabelecidos, como o Material Design 3 (Google, s.d.) ou Human Interface Guidelines (Apple Developer

Documentation, s.d.), para assegurar que a navegação seja previsível e intuitiva. • Modularização e reutilização: extrair componentes comuns (como botões personalizados ou estilos de texto) para arquivos de recursos (styles.xml) ou widgets customizados, facilitando atualizações globais no projeto. • Separação de responsabilidades: manter a lógica de comportamento isolada da definição declarativa do layout, preferencialmente utilizando padrões como o MVVM.

4 DESENVOLVIMENTO MULTIPLATAFORMA COM FLUTTER

4.1 Introdução ao Flutter e a engenharia multiplataforma

O Flutter é um framework de interface de usuário (UI Toolkit) de código aberto, criado pelo Google, projetado para a construção de aplicações compiladas nativamente para múltiplas plataformas – incluindo Android, iOS, Web e Desktop – a partir de uma única base de código. Diferente de abordagens híbridas tradicionais que utilizam pontes (bridges) de JavaScript, o Flutter destaca-se por sua capacidade de renderização direta. O framework utiliza o motor gráfico Impeller (ou Skia, em versões legadas) para desenhar cada pixel na tela, garantindo que a aplicação tenha um desempenho de 60 a 120 FPS

Em um comparativo estrutural do Flutter, um parecer técnico destaca a necessidade de contextualizar a escolha do Flutter frente a outros frameworks modernos, como o .NET MAUI. Enquanto o MAUI foca na abstração de componentes nativos (onde cada elemento XML mapeia para um controle específico de Android ou iOS), o Flutter prioriza a composição de widgets. Essa escolha técnica reduz inconsistências visuais entre plataformas, tornando o Flutter a opção preferencial para projetos que exigem alta produtividade e identidade visual única.

- Reutilização de lógica e UI: redução drástica do time-to-market ao compartilhar até 90% do código entre plataformas.
- Hot reload: recurso que permite visualizar alterações no código instantaneamente sem perder o estado da aplicação, otimizando o ciclo de desenvolvimento.
- Arquitetura baseada em widgets: tudo no Flutter é um widget, o que facilita a criação de interfaces complexas através da composição de pequenos blocos funcionais.

- Desempenho nativo: o código Dart é compilado para linguagem de máquina (AOT, ahead-of-time), eliminando gargalos de interpretação em tempo de execução.

4.2 Estrutura arquitetural de um aplicativo Flutter

A lógica de negócio deve estar isolada da interface do usuário e de detalhes de implementação, como bancos de dados ou APIs externas.

Na padronização de diretórios e camadas, é recomendada uma estrutura para projetos escaláveis que organiza o código em camadas lógicas. São elas:

- /domain (camada de domínio): contém as entidades de negócio e use cases. É a camada mais interna e não deve ter dependências externas.
- /data (camada de dados): responsável pela implementação de repositórios, fontes de dados (data sources) e modelos para serialização de JSON (DTOs).
- /presentation (camada de apresentação): inclui os widgets (UI) e os gerenciadores de estado (como Controllers, BLoCs ou ViewModels).
- /di (injeção de dependência): centraliza a configuração de instâncias e serviços, reduzindo o acoplamento entre as classes.

Quadro 6 – Organização

Padrão	Foco principal	Contexto de uso
MVC	Separação simples de view e lógica	Projetos de pequena escala ou protótipos
MVVM	Reatividade e desacoplamento da UI	Padrão ouro para Flutter com gerenciamento de estado
Clean Arch	Independência total de ferramentas externas	Aplicações empresariais robustas e de longa duração

4.3 Construção de interfaces baseada em widgets

No ecossistema Flutter, a interface do usuário é construída sob o paradigma de que tudo é um widget. Diferente do desenvolvimento Android tradicional, que separa a marcação (XML) da lógica (Java/Kotlin), o Flutter utiliza a composição declarativa em Dart para definir tanto a estrutura visual quanto o comportamento dos elementos. Os widgets funcionam como unidades básicas de construção que descrevem como a visualização deve ser apresentada em função de sua configuração e estado atual.

- Composição visual: permite aninhar elementos para criar layouts complexos sem a rigidez de hierarquias de herança profundas.
- Reutilização e customização: widgets

podem ser extraídos para classes independentes, promovendo a padronização e reduzindo a duplicação de código. • Acessibilidade e temas: o uso de widgets nativos do material design garante que a aplicação suporte automaticamente recursos de acessibilidade e modos escuro/claro.

```
import 'package:flutter/material.dart';

class LoginPage extends StatelessWidget {

  @override

  Widget build(BuildContext context) {

    return Scaffold(

      appBar: AppBar(title: Text('Acesso ao Sistema')),

      body: Padding(

        padding: EdgeInsets.all(16.0),

        child: Column(

          mainAxisAlignment: MainAxisAlignment.center,

          children: [

            // Widget de entrada de texto com estilo Material

            TextField(

              decoration: InputDecoration(

                labelText: 'E-mail',

                border: OutlineInputBorder(),

              ),

              keyboardType: TextInputType.emailAddress,

            ),

            SizedBox(height: 16), // Espaçamento entre componentes

            TextField(

              decoration: InputDecoration(

                labelText: 'Senha',

                border: OutlineInputBorder(),
```

```

    ),
    obscureText: true, // Segurança: máscara de senha
  ),
  SizedBox(height: 24),
  ElevatedButton(
    onPressed: () {},
    child: Text('ENTRAR'),
    style: ElevatedButton.styleFrom(minimumSize: Size(double.infinity, 50)),
  ),
],
),
),
);
}
}

```

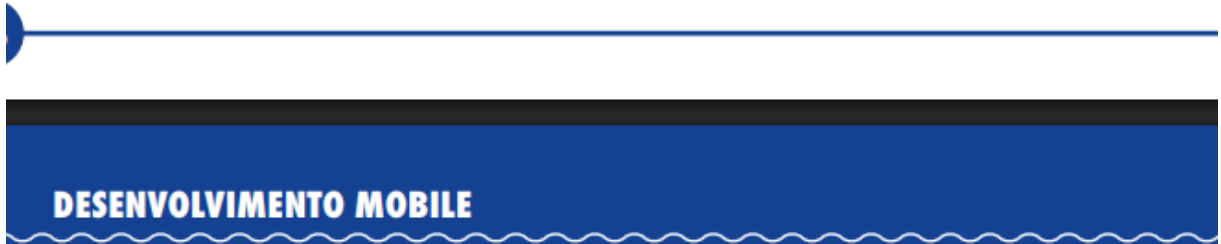
4.4 Layouts e organização de componentes

No Flutter, a organização espacial dos elementos baseia-se no modelo de caixa (box model) e em sistemas de coordenadas flexíveis. A estruturação de interfaces complexas exige o domínio de widgets de layout que gerenciam como os filhos (children) são distribuídos nos eixos vertical e horizontal.

Column: organização vertical

O widget column é utilizado para dispor múltiplos widgets em uma sequência vertical. Tecnicamente, ele é um widget de eixo principal vertical (main axis), onde o alinhamento é controlado pelas propriedades mainAxisAlignment (alinhamento vertical) e crossAxisAlignment (alinhamento horizontal dentro da coluna).

```
Column(
  mainAxisAlignment: MainAxisAlignment.center, // Centraliza verticalmente
  crossAxisAlignment: CrossAxisAlignment.stretch, // Expande os filhos horizontalmente
  children: [
    Text('Título do Formulário', style: TextStyle(fontWeight: FontWeight.bold)),
```



```
    TextField(decoration: InputDecoration(hintText: 'Digite seu usuário')),
    ElevatedButton(onPressed: () {}, child: Text('Confirmar')),
  ],
)
```

Row: organização horizontal

O widget row dispõe seus elementos filhos em uma linha horizontal. Assim como a column, ele utiliza propriedades de alinhamento, mas com os eixos invertidos: o mainAxis é o horizontal e o crossAxis é o vertical.

Container: widget de estilização e box model

- Padding e margin: controle de espaçamento interno e externo utilizando a classe EdgeInsets.
- Decoration: aplicação de cores de fundo, bordas arredondadas e sombras através da classe BoxDecoration.
- Constraints: definição de limites de largura e altura para garantir a responsividade.

Widget	Eixo principal (main)	Caso de uso profissional
Column	Vertical	Estruturação de formulários e listas de dados
Row	Horizontal	Menus de navegação e agrupamento de ícones e textos
Container	N/A (Box)	Criação de cards, aplicação de estilos e controle de margens

4.5 Responsividade e adaptação de interfaces em Flutter

No desenvolvimento multiplataforma, a responsividade transcende o simples ajuste de dimensões: trata-se de garantir a integridade da experiência do usuário em um ecossistema fragmentado que inclui smartphones compactos, tablets, dispositivos dobráveis e diferentes orientações de tela (portrait e landscape).

O Flutter aborda esse desafio através de um sistema de restrições (constraints) e ferramentas de consulta ao contexto do dispositivo. Diferente de valores estáticos, a responsividade técnica exige o uso de dimensões dinâmicas que reagem às variações do hardware em tempo real.

Para tal função, o MediaQuery é o recurso fundamental a fim de obter os dados do objeto window, fornecendo o contexto físico do dispositivo. Ele permite que o código Dart tome decisões lógicas baseadas em metadados da tela.

```
@override
Widget build(BuildContext context) {

  // Obtém informações da tela via InheritedWidget

  final mediaQueryData = MediaQuery.of(context);

  final double largura = mediaQueryData.size.width;

  final Orientation orientacao = mediaQueryData.orientation;
```

58

DESENVOLVIMENTO MOBILE

```
return Container(

  // Define a largura baseada em uma proporção da tela (ex: 80%)

  width: largura * 0.8,

  color: orientacao == Orientation.portrait ? Colors.blue : Colors.green,

  child: Text('Layout Adaptável'),

);

}
```

Para evitar erros de transbordamento (overflow) e garantir o preenchimento proporcional do espaço:

- Expanded: obriga o filho a preencher todo o espaço restante no eixo principal de uma column ou row.
- Flexible: semelhante ao expanded, mas permite que o filho seja menor que o espaço disponível, oferecendo maior controle sobre o fator de flexibilidade (flex factor).
- LayoutBuilder: widget essencial para a criação de layouts adaptativos. Ele fornece as BoxConstraints do pai, permitindo que o desenvolvedor altere a estrutura da árvore de widgets (como mudar de uma column para uma row) dependendo da largura disponível.

Quadro 8 – Adaptação de interface

Técnica	Aplicação técnica	Benefício na UX
Dimensionamento percentual	Uso de MediaQuery para definir tamanhos relativos	Consistência em diferentes densidades de pixels
Layouts condicionais	Uso de LayoutBuilder para alternar entre UI de Tablet/Mobile	Otimização do espaço útil em telas maiores
Flexibilidade dinâmica	Uso de Expanded e Spacer em eixos flexíveis	Prevenção da quebra da interface e de elementos cortados

4.6 Gerenciamento de estado e reatividade

No desenvolvimento com Flutter, o estado (state) é definido tecnicamente como qualquer dado que pode mudar durante o ciclo de vida de um widget e que, ao sofrer alteração, exige a reconstrução da interface para refletir o novo valor.

- StatelessWidget: representa widgets imutáveis. Uma vez construídos, suas propriedades não mudam. São ideais para elementos estáticos, como ícones ou rótulos de texto fixos, pois consomem menos recursos de memória.
- StatefulWidget: widgets que mantêm um objeto de estado separado, permitindo que a interface seja reconstruída dinamicamente. São essenciais para campos de formulário, animações e telas que dependem de dados assíncronos.

Método `setState()` é o gatilho básico da reatividade no Flutter. Ele notifica o framework de que o estado interno de um objeto foi alterado, inserindo o widget na lista de “sujos” (dirty widgets) para que o motor de renderização execute o método `build()` novamente.

```
// Exemplo de contador utilizando StatefulWidget

class ContadorWidget extends StatefulWidget {
  @override
  _ContadorWidgetState createState() => _ContadorWidgetState();
}

class _ContadorWidgetState extends State<ContadorWidget> {
  int _contador = 0;

  void _incrementar() {
    // Notifica o framework sobre a mudança de estado
    setState(() {
```

0

DESENVOLVIMENTO MOBILE

```
      _contador++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return ElevatedButton(
      onPressed: _incrementar,
      child: Text('Cliques: $_contador'),
    );
  }
}
```

4.6.1 Abordagens de gerenciamento de estado moderno

Embora o `setState()` seja eficaz para estados locais e simples, o parecer técnico aponta que aplicações de nível superior exigem padrões que permitam o desacoplamento entre a lógica de negócio e a interface (UI).

- **Provider/Riverpod**: baseados no conceito de `InheritedWidget`, permitem que os dados fluam pela árvore de widgets de forma controlada, facilitando a manutenção e os testes unitários.
- **BLoC**: utiliza fluxos de dados (streams) para separar totalmente a lógica da interface, sendo o padrão mais recomendado para arquiteturas robustas e equipes grandes.

Quadro 9 – Gerenciamento de estado

Solução	Escopo técnico	Recomendação de uso
<code>setState()</code>	Local (widget)	Protótipos e componentes isolados
Provider	Global/injeção	Aplicações de médio porte e MVM
BLoC	Fluxos/streams	Aplicações empresariais e clean architecture

É importante ressaltar que o uso indiscriminado do `setState()` em widgets de alto nível (como no topo da árvore) pode causar reconstruções desnecessárias em toda a aplicação, impactando o desempenho da CPU e o consumo de bateria. Por isso, prioriza-se o levantamento de estado (lifting state up) e o uso de gerenciadores que atualizem apenas os nós específicos da árvore de widgets.

Implementação de um contador reativo

```
import 'package:flutter/material.dart';

// O widget pai é imutável
class ContadorPratico extends StatefulWidget {

  @override
  _ContadorPraticoState createState() => _ContadorPraticoState();
}

// O objeto de estado contém a lógica mutável
class _ContadorPraticoState extends State<ContadorPratico> {

  int _contador = 0; // Estado da aplicação

  void _incrementarContador() {

    // setState notifica o framework para executar o método build() novamente

    setState(() {

      _contador++;

    });

  }

  @override
  Widget build(BuildContext context) {

    return Column(

      mainAxisAlignment: MainAxisAlignment.center,

      children: [

        Text(

          'Quantidade de cliques:',

          style: TextStyle(fontSize: 18),


```

62

DESENVOLVIMENTO MOBILE

```
    ),

    Text(

      '$_contador', // Interpolação de string para exibir o estado

      style: TextStyle(fontSize: 40, fontWeight: FontWeight.bold, color:

Colors.blue),

    ),

    SizedBox(height: 20),

    ElevatedButton.icon(

      onPressed: _incrementarContador,

      icon: Icon(Icons.add),

      label: Text('Incrementar'),

    ),

  ],

);

}
```

- **Separação de identidade:** o widget é dividido em duas classes: ContadorPratico (o widget em si) e _ContadorPraticoState (onde reside o estado).

Observação: Em aplicações de larga escala, o estado do contador poderia ser gerenciado externamente por um Provider ou BLoC, permitindo que o valor fosse

compartilhado entre diferentes telas sem a necessidade de passar parâmetros manualmente via construtor.

4.7 Garantia de qualidade e testes de interface

A confiabilidade de uma aplicação móvel é assegurada por meio de uma estratégia rigorosa de testes automatizados. No ecossistema Flutter, o framework fornece ferramentas nativas para validar desde pequenas unidades de lógica até a interação completa do usuário com a interface.

- Testes unitários: validam funções, métodos ou classes isoladas. São extremamente rápidos e focam na lógica de negócio pura.
- Testes de widget (componente): o nível intermediário, que verifica se o widget renderiza corretamente e responde às interações conforme o esperado, sem a necessidade de um emulador completo.
- Testes de integração: validam o fluxo completo da aplicação em um dispositivo real ou emulador, verificando a comunicação entre diferentes módulos e serviços externos.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:meu_projeto/main.dart';

void main() {

  testWidgets('Validação de incremento do contador', (WidgetTester tester) async {

    // Constrói a aplicação no ambiente de teste
    await tester.pumpWidget(MeuApp());

    // Verifica se o estado inicial é zero
    expect(find.text('0'), findsOneWidget);

    // Simula uma interação de toque no botão
    await tester.tap(find.byIcon(Icons.add));

    // Reconstrói o widget após a mudança de estado
    await tester.pump();

    // Verifica se o valor foi atualizado para 1
    expect(find.text('1'), findsOneWidget);

  });
}
```

4.8 Boas práticas e governança de código no Flutter

- Organização e reutilização de widgets: a modularização é a chave para evitar a duplicação de código (DRY – “don’t repeat yourself”). – Widget refactoring: widgets com mais de 60 linhas de código devem ser refatorados em componentes menores e especializados. – Imutabilidade: sempre que possível, utilize StatelessWidget e marque construtores como const. Isso permite que o Flutter reutilize instâncias na memória, otimizando drasticamente a renderização.
- Gerenciamento de estado e arquitetura: a escolha do gerenciamento de estado deve estar alinhada à complexidade do negócio: – Separação de lógica (logic decoupling): a lógica de negócio nunca deve residir no método build(). Utilize padrões como MVVM ou BLoC para que a interface apenas reaja a mudanças de estado. – Injeção de dependência: utilize ferramentas como GetIt ou Provider para gerenciar as instâncias de serviços e repositórios, facilitando a inversão de controle e os testes unitários.
- Otimização de performance: para manter a fluidez de 60 FPS, o desenvolvedor deve atentar-se para: – Uso eficiente do RepaintBoundary: isole widgets que sofrem atualizações frequentes (como animações) para que o Flutter não precise redesenhar a tela inteira. – Tratamento assíncrono: garanta que operações pesadas (I/O, processamento de dados) sejam executadas em Isolates ou via Futures/Streams para não bloquear a main thread (UI Thread).

Unidade III

5 INTEGRAÇÃO DE DADOS E FUNCIONALIDADES AVANÇADAS

5.1 Consumo de dados e arquitetura de integração com APIs

No ecossistema de desenvolvimento móvel, as APIs (application programming interfaces) atuam como a camada de abstração que permite a interoperabilidade

entre o dispositivo cliente e servidores remotos. APIs baseadas em REST utilizam o protocolo HTTP de forma desacoplada

- **Requisição (request):** o aplicativo móvel (cliente) envia um pacote de dados via HTTP para um endpoint específico (como GET/clima/sao-paulo).
- **Processamento:** o servidor processa a solicitação, consulta bancos de dados e aplica regras de negócio.
- **Resposta (response):** o servidor devolve um código de status (por exemplo, 200 OK) acompanhado de um corpo de dados, geralmente em formato JSON.
- **Renderização:** o aplicativo interpreta os dados recebidos e atualiza a árvore de widgets de forma reativa.

5.2 Arquitetura cliente-servidor e o papel estratégico das APIs

Neste modelo, o processamento é distribuído entre dois componentes principais: o cliente (a aplicação instalada no dispositivo móvel), responsável pela lógica de apresentação e interação; e o servidor (composto por APIs, serviços de nuvem e bancos de dados), responsável pela persistência, regras de negócio complexas e segurança.

- **Sincronização multiplataforma:** garantia de que o usuário acesse os mesmos dados em diferentes dispositivos (Android, iOS ou Web) de forma agnóstica.
- **Autenticação centralizada:** implementação de protocolos de segurança como OAuth 2.0 e JWT para validar a identidade do usuário de forma segura e persistente.
- **Eficiência de manutenção:** alterações nas regras de negócio ou nos dados podem ser feitas diretamente no servidor, impactando instantaneamente todos os clientes sem a necessidade de novos builds nas lojas de aplicativos

5.3 Comunicação assíncrona e protocolo HTTP

Métodos HTTP e semântica de operações

- **GET (leitura):** solicita a representação de um recurso. Não deve causar efeitos colaterais no servidor.
- **POST (escrita):** submete dados para serem processados, resultando frequentemente na criação de um novo recurso.
- **PUT (atualização):** substitui todas as representações atuais do recurso de destino pelos dados da requisição.
- **DELETE (remoção):** exclui o recurso especificado no servidor.

```
import 'package:http/http.dart' as http;
import 'dart:convert';

// Exemplo de busca de dados (GET) com tratamento de estado
Future<void> buscarPerfil(int id) async {
  try {
    final response = await http.get(
      Uri.parse('https://api.exemplo.com/usuarios/$id'),
      headers: {'Authorization': 'Bearer <token>'},
    ).timeout(Duration(seconds: 10)); // Controle de Timeout
    if (response.statusCode == 200) {
      var dados = jsonDecode(response.body);
      print('Usuário: ${dados['nome']}');
    } else {
      throw Exception('Erro ao carregar perfil: ${response.statusCode}');
    }
  }
}
```

Unidade III

```
} catch (e) {
  print('Falha na comunicação: $e'); // Tratamento de erros por camada
}
}
```

Quadro 11 – Códigos de status e tratamento em mobile

Código	Classe	Significado técnico	Ação sugerida no app
200/201	Sucesso	Requisição processada com êxito	Atualizar UI com os dados recebidos
400	Erro do cliente	Requisição malformada ou erro de validação	Exibir alerta de erro ao usuário
401/403	Erro de permissão	Token ausente, expirado ou inválido	Redirecionar para tela de login (refresh token)
404	Não encontrado	O endpoint ou recurso não existe	Exibir tela de "Conteúdo não encontrado"
500	Erro do servidor	Falha interna no processamento do backend	Sugerir tentativa posterior ao usuário

5.4 Manipulação e serialização de dados (JSON)

```
import 'dart:convert';

void processarResposta(String responseBody) {

    // Converte a String JSON em um Map dinâmico

    final Map<String, dynamic> dados = jsonDecode(responseBody);
```

Unidade III

```
// Acesso seguro utilizando o operador Null-Coalescing (??)

final String titulo = dados['titulo'] ?? 'Título indisponível';

print('Processando: $titulo');

}
```

Modelagem de dados e DTOs (data transfer objects)

Para projetos de nível profissional, o parecer técnico veda o uso exclusivo de Maps genéricos (como dados['campo']), pois essa prática é propensa a erros de digitação e dificulta a manutenção. A boa prática de engenharia recomenda a criação de classes de modelo com construtores do tipo factory.

```
class Post {

    final int id;

    final String titulo;

    Post({required this.id, required this.titulo});

    // Factory constructor para converter JSON em objeto Post

    factory Post.fromJson(Map<String, dynamic> json) {

        return Post(

            id: json['id'] as int,

            titulo: json['titulo'] as String? ?? 'Sem título',

        );

    }

}
```

5.5 Tratamento de exceções e resiliência em sistemas móveis

```
import 'package:http/http.dart' as http;

import 'dart:async'; // Necessário para TimeoutException

Future<void> realizarIntegracaoSegura() async {

  try {

    final url = Uri.parse("https://api.exemplo.com/v1/posts/1");

    // Implementação de Timeout para evitar processos "zumbis"

    final response = await http.get(url).timeout(const Duration(seconds: 10));

    // Validação da Semântica HTTP

    if (response.statusCode == 200) {

      final dados = jsonDecode(response.body);

      // Processamento lógico...

    } else {

      // Tratamento de erros de negócio (4xx e 5xx)

      _logError("Erro de Servidor: ${response.statusCode}");

    }

  } on TimeoutException catch (_) {
```

8

Unidade III

```
_notificarUsuario("A conexão expirou. Verifique sua rede.");

} on SocketException catch (e) {

  _notificarUsuario("Sem conectividade com a internet.");

} catch (e) {

  // Captura de exceções genéricas ou de parsing

  _logError("Falha inesperada: $e");

}

}
```

- Exceções de rede (SocketException): causadas por ausência de sinal ou DNS inválido. A solução envolve o uso de bibliotecas de monitoramento de conectividade (Connectivity Plus).
- Exceções de protocolo (Status 4xx/5xx): o servidor respondeu, mas não pôde processar o pedido. Exige lógica de negócio para redirecionamento (por exemplo, 401 para login) ou mensagens de erro amigáveis.
- Exceções de tempo (TimeoutException): ocorrem quando o servidor demora a responder. É

essencial para evitar o bloqueio da interface e o consumo excessivo de recursos. •
Exceções de dados (FormatException): ocorrem quando o JSON recebido está corrompido ou malformatado.

5.6 Exemplo prático integrado: consumo de endpoints REST

```
import 'package:http/http.dart' as http;
import 'dart:convert';
import 'dart:io';

/// Realiza o consumo de uma lista de usuários de forma assíncrona.
Future<void> carregarUsuarios() async {
  try {
    // Passo 1: Definição do Endpoint e conversão para Objeto Uri
    final url = Uri.parse('https://jsonplaceholder.typicode.com/users');

    // Passo 2: Execução da requisição GET com await
    final response = await http.get(url).timeout(const Duration(seconds: 15));

    // Passo 3: Validação da Semântica HTTP (Status 200 OK)
    if (response.statusCode == 200) {
      // Passo 4: Desserialização do corpo da resposta (String para List)
      final List<dynamic> dados = jsonDecode(response.body);

      // Passo 5: Validação de estrutura e acesso seguro (Null Safety)
      if (dados.isNotEmpty) {
```

Unidade III

```
    final String nomePrimeiroUsuario = dados[0]['name'] ?? 'Usuário sem
nome';

    print('Integração bem-sucedida: $nomePrimeiroUsuario');
  }
} else {
  print('Falha no servidor: Código ${response.statusCode}');
}
} on SocketException {
  print('Erro de rede: Certifique-se de que o dispositivo está conectado.');
```

```
} catch (e) {
  // Captura de exceções genéricas (ex: falha no parsing do JSON)
  print('Erro inesperado na integração: $e');
}
}
```

6 ARMAZENAMENTO E RECURSOS DO DISPOSITIVO

6.1 Estratégias de persistência de dados local

Uma das estratégias a serem implementadas ocorre com a persistência de chave-valor com SharedPreferences. O SharedPreferences é uma API que permite o armazenamento de dados primitivos (strings, inteiros, booleanos) em um formato de mapa persistente. Tecnicamente, no Android, esses dados são gravados em arquivos XML privados, enquanto no iOS utilizam o UserDefaults. Veja o exemplo de código a seguir.

```
import 'package:shared_preferences/shared_preferences.dart';

/// Persiste as preferências de interface do usuário

Future<void> salvarConfiguracoes(String tema, bool notificacoes) async {

  // Obtém a instância do repositório local

  final SharedPreferences prefs = await SharedPreferences.getInstance();

  // Escrita assíncrona de dados primitivos

  await prefs.setString('tema_app', tema);

  await prefs.setBool('status_notificacoes', notificacoes);

}

/// Recupera dados persistidos com tratamento de nulidade

Future<void> carregarConfiguracoes() async {

  final prefs = await SharedPreferences.getInstance();

  // O uso do operador '?' indica que o valor pode ser nulo (Null Safety)

  final String? temaAtivo = prefs.getString('tema_app');

  // Definição de valor padrão caso a chave não exista (Null-coalescing)

  print('Tema recuperado: ${temaAtivo ?? "Padrão do Sistema"}');

}
```

Informações como tokens JWT, senhas ou dados bancários devem ser obrigatoriamente protegidos por camadas de criptografia de hardware. No Flutter, utiliza-se o plugin flutter_secure_storage, que implementa:

- Android:

armazenamento no KeyStore com criptografia AES. • iOS: persistência no KeyChain, garantindo que os dados não sejam acessíveis mesmo em dispositivos com jailbreak.

Quadro 13 – Comparativo de persistência local

Tecnologia	Tipo de dado	Segurança nativa	Recomendação técnica
SharedPreferences	Primitivos (key-value)	Baixa (texto puro)	Preferências de UI e flags simples
Secure Storage	Strings criptografadas	Alta (hardware)	Tokens de API e credenciais
SQLite (sqflite)	Dados estruturados	Média (extensível)	Históricos, cache de API e dados complexos

6.2 Persistência estruturada com SQLite

SharedPreferences é limitado a metadados simples, aplicações que gerenciam volumes expressivos de informação ou que exigem relacionamentos complexos utilizam o SQLite. Trata-se de um motor de banco de dados relacional leve, baseado em arquivos, que opera de forma embarcada (embedded) no dispositivo móvel.

- DAO (Data Access Object): classe responsável exclusivamente por executar os comandos SQL (INSERT, SELECT etc.) no banco.
- Repository pattern: atua como um mediador entre a fonte de dados (SQLite ou API) e a lógica de negócio, decidindo de onde os dados devem ser recuperados.

```

import 'package:sqflite/sqflite.dart';
import 'package:path/path.dart';

class TarefaDAO {

  // Inicialização e criação da estrutura de tabelas

  Future<Database> _getDatabase() async {

    final caminho = join(await getDatabasesPath(), 'app_tarefas.db');

    return openDatabase(caminho, version: 1, onCreate: (db, v) {

      return db.execute('CREATE TABLE tarefas (id INTEGER PRIMARY KEY, titulo
      TEXT)');

    });

  }

  // Operação de Inserção (Create)

  Future<int> inserir(Map<String, dynamic> dados) async {

    final db = await _getDatabase();

    return await db.insert('tarefas', dados, conflictAlgorithm: ConflictAlgorithm.
    replace);

  }

  // Operação de Consulta (Read)

  Future<List<Map<String, dynamic>>> listar() async {

    final db = await _getDatabase();

    return await db.query('tarefas', orderBy: 'id DESC');

  }

}

```

6.3 Integração com recursos de hardware e sensores

Modernamente, tanto o Android quanto o iOS operam sob o modelo de permissões em tempo de execução (runtime permissions). Isso significa que declarar a necessidade de um recurso nos arquivos de manifesto (AndroidManifest.xml e Info.plist) é apenas o primeiro passo; o aplicativo deve obrigatoriamente solicitar o consentimento explícito do usuário durante o uso.

```
import 'package:geolocator/geolocator.dart';

/// Gerencia a obtenção da localização com verificação de privilégios
Future<void> obterLocalizacaoAtual() async {

  bool servicoAtivo;
```

Unidade III

```
LocationPermission permissao;

// 1. Verifica se o hardware de GPS está habilitado
servicoAtivo = await Geolocator.isLocationServiceEnabled();

if (!servicoAtivo) return Future.error('O serviço de localização está
desativado.');
```

```
// 2. Verifica o status atual da permissão
permissao = await Geolocator.checkPermission();

if (permissao == LocationPermission.denied) {

  // Solicita autorização formal ao usuário
  permissao = await Geolocator.requestPermission();

  if (permissao == LocationPermission.denied) {

    return Future.error('Permissão negada pelo usuário.');
```

```
  }
}

// 3. Acesso ao recurso de hardware após validação
Position posicao = await Geolocator.getCurrentPosition(

  desiredAccuracy: LocationAccuracy.high

);

print('Coordenadas: ${posicao.latitude}, ${posicao.longitude}');
```

```
}
```

- Geolocalização (GPS): essencial para logística, geofencing (cercas virtuais) e serviços baseados em contexto local.
- Captura de imagem e biometria: utilizados para processos de Know Your Customer (KYC), autenticação segura e redes sociais.
- Sensores de movimento: utilizados em aplicações de saúde e acessibilidade para detectar quedas ou orientar a interface.

- Android (AndroidManifest.xml): inclusão de tags para declarar o uso do hardware.
- iOS (Info.plist): inclusão de chaves como `NSLocationWhenInUseUsageDescription`, contendo a justificativa clara do porquê o aplicativo necessita do dado. A ausência dessa justificativa resulta em rejeição automática na App Store.

Quadro 15 – Matriz de permissões e justificativas

Recurso	Declaração técnica	Justificativa de UX (exemplo)
Localização	<code>ACCESS_FINE_LOCATION</code>	Necessário para encontrar pontos de entrega próximos
Câmera	<code>CAMERA</code>	Utilizada para digitalizar códigos de barras e documentos
Arquivos	<code>READ_EXTERNAL_STORAGE</code>	Permite o upload de fotos de perfil da sua galeria

6.4 Arquitetura offline-first e governança de permissões

O paradigma offline-first indica que a aplicação deve priorizar o armazenamento local (SQLite/Cache) e realizar a sincronização com o servidor de forma assíncrona e transparente. Isso garante consistência de informações e uma experiência de usuário sem interrupções, tendo, assim, diversas vantagens:

- Continuidade de serviço: o usuário pode realizar transações (como cadastros ou pedidos) mesmo sem internet.
- Eficiência de banda: sincronizações automáticas podem ser programadas para ocorrer apenas via Wi-Fi, reduzindo o consumo de dados móveis.
- Performance percebida: a leitura de dados locais é instantânea, eliminando a percepção de latência de rede.

A utilização de recursos sensíveis (como GPS, câmera, microfone etc.) é estritamente regulada pelos sistemas operacionais Android e iOS. O desenvolvedor deve implementar uma lógica de solicitação justificada, onde o usuário compreende o valor da concessão do acesso.

```

import 'package:geolocator/geolocator.dart';
import 'package:flutter/material.dart';

class LocalizacaoService {

  /// Solicita autorização e recupera a posição geográfica
  Future<void> gerenciarPermissaoEContexto(BuildContext context) async {

    LocationPermission permission;

    // 1. Verifica status atual da permissão
    permission = await Geolocator.checkPermission();

    if (permission == LocationPermission.denied) {

      // 2. Solicitação formal de acesso (Runtime Permission)
      permission = await Geolocator.requestPermission();

      if (permission == LocationPermission.denied) {

        _notificarErro(context, 'Acesso à localização negado.');
```

```

        return;
      }
    }

    if (permission == LocationPermission.deniedForever) {

      _notificarErro(context, 'Permissão permanentemente negada. Altere nas
configurações.');
```

```

      return;
    }

    // 3. Execução do recurso após validação
    Position position = await Geolocator.getCurrentPosition();

    print('Localização obtida: ${position.latitude}, ${position.longitude}');
```

```

  }
}

```

```

void _notificarErro(BuildContext context, String msg) {

  ScaffoldMessenger.of(context).showSnackBar(SnackBar(content: Text(msg)));

}

}

```

- **Android (AndroidManifest.xml):** é necessário utilizar o código a seguir:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

- **iOS (Info.plist):** é obrigatório incluir a chave `NSLocationWhenInUseUsageDescription` com uma mensagem clara para o usuário. A ausência desta descrição é motivo de rejeição sumária na loja.



6.5 Sincronização de dados e testes de integração

A sincronização é o processo de coordenação de dados entre o armazenamento local (SQLite/Cache) e o servidor remoto. Em sistemas de nível superior, utilizamos o conceito de consistência eventual, onde o aplicativo garante que, embora os dados possam divergir temporariamente devido à falta de conexão, eles serão reconciliados assim que a conectividade for restabelecida.

- **Captura local:** a ação do usuário é persistida imediatamente no banco de dados local (por exemplo, SQLite).
- **Fila de sincronização (queue):** a operação é enviada para uma fila de processamento em segundo plano (background task).
- **Verificação de conectividade:** o sistema monitora o status da rede.
- **Reconciliação:** ao detectar internet, o app envia o payload para a API e trata possíveis conflitos de versão (por exemplo, o mesmo dado alterado em dois dispositivos diferentes).

No contexto de APIs, os testes de integração verificam se o contrato entre o frontend e o backend está sendo respeitado, validando a serialização JSON e o tratamento de códigos de status HTTP.

```

import 'package:flutter_test/flutter_test.dart';

import 'package:http/http.dart' as http;

import 'package:mockito/mockito.dart';

// Simulação (Mock) do cliente HTTP para testes controlados
class MockClient extends Mock implements http.Client {}

void main() {

  test('Deve retornar um Objeto de Usuário se a chamada HTTP for 200', () async {

    final client = MockClient();

    // Configura o Mock para retornar sucesso e um JSON válido
    when(client.get(Uri.parse('https://api.exemplo.com/user')))

      .thenAnswer((_) async => http.Response('{ "name": "Alex" }', 200));

    final response = await client.get(Uri.parse('https://api.exemplo.com/user'));

    expect(response.statusCode, 200);

    expect(response.body, contains('Alex'));

  });

  test('Deve lançar exceção em caso de erro 404', () async {

    final client = MockClient();

    when(client.get(any))

      .thenAnswer((_) async => http.Response('Not Found', 404));

    final response = await client.get(Uri.parse('https://api.exemplo.com/user'));

    expect(response.statusCode, 404);

  });

}

```

Unidade IV

7 ENGENHARIA E ARQUITETURA DE APLICAÇÕES MOBILE

7.1 Estrutura de aplicações mobile modernas e padronização arquitetural

Aplicações modernas são sistemas distribuídos que exigem uma orquestração precisa entre a interface (UI), as regras de negócio e a camada de dados. Uma aplicação profissional deve ser segmentada nos seguintes pilares:

- Interface (presentation layer): camada responsável pela renderização visual e captura de eventos do usuário.
- Lógica de negócio (domain layer): contém as regras puras da aplicação, independente de frameworks ou fontes de dados.
- Persistência e integração (data layer): gerencia o ciclo de vida dos dados, seja via consumo de APIs RESTful ou armazenamento local em SQLite.
- Gerenciamento de estado: mecanismo que sincroniza a lógica de negócio com a interface, garantindo que o usuário visualize sempre a informação mais atualizada.



Padrões arquiteturais avançados: MVVM e clean architecture

- MVVM: foca na ligação de dados (data binding) entre a View e o ViewModel, removendo a lógica de estado da interface.
- Clean architecture: proposta por Martin (2019), esta arquitetura organiza o código em círculos concêntricos, onde as dependências apontam apenas para o centro (as regras de negócio), tornando o sistema independente de agentes externos.
- Repository pattern: atua como uma fachada que abstrai se os dados estão vindo de um cache local ou de uma requisição de rede, simplificando a lógica do controlador.
- Injeção de dependência: técnica que permite fornecer as instâncias de classes necessárias (como serviços de API) dinamicamente, facilitando a criação de testes unitários e de integração.

7.2 Engenharia de navegação e fluxo de estados

O framework Flutter implementa a navegação baseada no conceito de LIFO (Last-In, First-Out), onde as interfaces são gerenciadas em uma pilha (stack) que possui duas ferramentas:

- Push: adiciona uma nova rota ao topo da pilha, ocultando a interface anterior, mas mantendo seu estado em memória.
- Pop: remove a interface do topo, retornando o controle para a rota imediatamente inferior na pilha.

```
// Navegação para uma nova rota com passagem de contexto
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => const DetalhesProdutoPage(),
  ),
);

// Encerramento da rota atual e retorno na pilha
Navigator.pop(context);
```

O código a seguir apresenta a configuração de rotas nomeadas:

```
MaterialApp(
  initialRoute: '/',
  routes: {
    '/': (context) => const LoginPage(),
    '/home': (context) => const HomePage(),
    '/perfil': (context) => const PerfilUsuarioPage(),
```



```
  },
);

// Invocação da rota via identificador de string
Navigator.pushNamed(context, '/home');
```

Além disso, uma arquitetura profissional deve prever fluxos de navegação que respeitem as regras de negócio e a experiência do usuário. Em um cenário de autenticação, por exemplo, o fluxo segue uma lógica de validação rigorosa:

- Intercepção: o sistema verifica o estado de autenticação (Token JWT).
- Redirecionamento: se o token for inválido, a pilha é limpa e o usuário é enviado para a rota de /login.
- Transição de contexto: após o sucesso, a navegação ocorre para a /home, injetando os dados do usuário no estado global da aplicação.

7.3 Implementação de sistemas de autenticação e gestão de sessão

A autenticação é um dos pilares críticos da engenharia de software mobile, envolvendo não apenas a interface de usuário (front end), mas a segurança da comunicação e a integridade do armazenamento de segredos.

O fluxo típico em uma arquitetura moderna utiliza tokens de acesso (JWT) e segue as seguintes etapas:

- Entrada de credenciais: o usuário fornece identificador e

segredo via interface protegida. • Transmissão segura: os dados são enviados via HTTPS para o provedor de identidade. • Emissão de tokens: após a validação, o servidor retorna um access token (curta duração) e um refresh token (longa duração).

• Persistência segura: o aplicativo armazena os tokens em áreas criptografadas do dispositivo (KeyStore/KeyChain). • Autorização: o token é injetado nos cabeçalhos (headers) de todas as requisições subsequentes para validar a autoridade do usuário.

```
// Implementação de campos de entrada com foco em UX e Segurança
Column(
  children: [
    // Campo de identificação do usuário
    TextField(
      controller: usuarioController,
      decoration: const InputDecoration(labelText: 'E-mail ou Usuário'),
      keyboardType: TextInputType.emailAddress,
    ),
    // Campo de segredo com mascaramento de caracteres
    TextField(
      controller: senhaController,
      obscureText: true, // Propriedade essencial para segurança visual
      decoration: const InputDecoration(labelText: 'Senha'),
    ),
    // Acionador da lógica de autenticação
    ElevatedButton(
      onPressed: () => _processarLogin(context),
      child: const Text('Autenticar'),
    ),
  ],
)
```

Quando o access token expira, o aplicativo deve ser capaz de solicitar um novo par de chaves de forma transparente para o usuário, utilizando o refresh token armazenado de forma segura. Isso mantém a sessão ativa sem exigir logins repetitivos, elevando a experiência do usuário sem comprometer a segurança.

Quadro 16 – Boas práticas de segurança em autenticação

Prática	Descrição técnica	Relevância
Criptografia em repouso	Uso de flutter_secure_storage para gravar tokens	Impede acesso aos dados por outros apps
SSL Pinning	Validação do certificado do servidor no app	Evita ataques de Man-in-the-Middle (MitM)
Obscure text	Propriedade nativa para ocultar a digitação de senhas	Protege contra observação direta (ombro)
Token expiry	Implementação de lógica de expiração e renovação	Limita a janela de uso de um token roubado

7.4 Engenharia de validação e integridade de dados

A validação de dados em aplicações móveis é uma camada crítica de defesa e usabilidade. Tecnicamente, ela atua em duas frentes: sanitização, para garantir que apenas dados no formato correto sejam enviados ao servidor (reduzindo o custo de processamento e latência); e feedback de interface, para orientar o usuário em tempo real.

No ecossistema Flutter, a abordagem profissional utiliza o widget Form com uma GlobalKey. Essa estrutura permite validar múltiplos campos simultaneamente e gerenciar o estado visual de erro de forma automática.

```
final _formKey = GlobalKey<FormState>(); // Chave global para controle do estado do formulário
```

```
Form(
```

```
  key: _formKey,
```

```
  child: Column(
```

```
    children: [
```

```
      TextFormField(
```

```
        controller: usuarioController,
```

```
        decoration: const InputDecoration(labelText: 'Usuário'),
```

```
        // Lógica de validação integrada
```

```
        validator: (value) {
```

```
          if (value == null || value.isEmpty) {
```

```
            return 'Por favor, insira seu identificador.';
```

```
          }
```

```
          return null;
```

```
        },
```

```
      ),
```

```
      TextFormField(
```

```
        controller: senhaController,
```

```
        obscureText: true,
```

```
        decoration: const InputDecoration(labelText: 'Senha'),
```

```
        validator: (value) {
```

```
          if (value != null && value.length < 6) {
```

```
            return 'A senha deve conter no mínimo 6 caracteres.';
```

```
          }
```

```

        return null;
    },
),
ElevatedButton(
    onPressed: () {
        // Dispara a validação de todos os campos do formulário
        if (_formKey.currentState!.validate()) {
            _realizarLoginProfissional();
        }
    },
    child: const Text('Entrar'),
),
],
),
)

```

O uso de `SnackBars` é recomendado para mensagens contextuais e temporárias que não exigem a interrupção total da tarefa (diferente de caixas de diálogo modais):

```

ScaffoldMessenger.of(context).showSnackBar(
    const SnackBar(
        content: Text('Credenciais inválidas. Verifique os dados e tente novamente.'),
        backgroundColor: Colors.redAccent,
        behavior: SnackBarBehavior.floating, // Melhoria de UX para dispositivos modernos
    ),
);

```

Quadro 17 – Matriz de validação e impacto

Tipo de validação	Exemplo	Impacto em ADS
Sintática	Máscara de CPF, e-mail, telefone	Garante integridade na persistência
Semântica	Senha com requisitos de complexidade	Aumenta o nível de segurança do sistema
De negócio	Verificação de disponibilidade de login	Depende de integração síncrona com API

Observação: Em arquiteturas escaláveis, as regras de validação complexas devem residir na camada de domínio (domain layer) ou no ViewModel, permitindo que a mesma lógica seja testada unitariamente sem a necessidade de instanciar a interface gráfica.

7.5 Arquitetura de software: organização e separação de responsabilidades

A arquitetura deve ser organizada em camadas com responsabilidades bem definidas, permitindo que cada componente evolua de forma independente.

O modelo de camadas (SoC – separation of concerns)

Três domínios principais:

- Camada de modelos (models): representa as entidades de dados e as regras de negócio puras. No Dart, utilizamos classes para definir a estrutura dos objetos e métodos de serialização (por exemplo, fromJson e toJson).
- Camada de serviços (services): responsável pela comunicação com o mundo externo, como clientes HTTP (APIs) e provedores de hardware. É a camada onde ocorrem as operações de I/O assíncronas.
- Camada de visualização (views): composta exclusivamente por widgets. Sua única função é renderizar dados e capturar eventos do usuário, delegando qualquer processamento lógico para camadas superiores.

Para abstrair a origem dos dados, utiliza-se o repository pattern. Este padrão atua como um mediador: a view solicita um dado ao repositório, e este decide se deve buscá-lo em um cache local (SQLite) ou em uma API remota.

```
// 1. Camada de Modelo: Definição da Entidade

class Usuario {

    final int id;

    final String nome;

    Usuario({required this.id, required this.nome});

}

// 2. Camada de Serviço: Integração Técnica

class AuthService {

    Future<Map<String, dynamic>> autenticarRemoto(String login, String senha)
    async {

        // Lógica de comunicação com API via HTTP...

        return {'status': 200, 'nome': 'Alex'};

    }

}

// 3. Camada de Repositório: Orquestração e Abstração

class UsuarioRepository {

    final AuthService _service = AuthService();

    Future<Usuario> login(String user, String pass) async {

        final response = await _service.autenticarRemoto(user, pass);

        return Usuario(id: 1, nome: response['nome']);

    }

}
```

- a view captura o clique e notifica o controller ou viewmodel;
- o controller aciona o repositório;
- o repositório utiliza os services para obter dados e retorna um model;
- a view reage à mudança do dado e se atualiza.

7.6 Engenharia de gerenciamento de estado

O estado refere-se a qualquer dado que, ao ser alterado, exige uma atualização na interface do usuário (UI). O gerenciamento eficiente desse estado é o que define a performance e a manutenibilidade de uma aplicação complexa. Conforme o volume de dados e interações cresce, a simples atualização local torna-se insuficiente, exigindo arquiteturas que separem a lógica de negócio do ciclo de vida dos widgets.

- Estado local (setState): adequado para componentes isolados (como ligar/desligar um switch ou controlar um campo de texto). É simples, mas gera alto acoplamento, dificultando o compartilhamento de dados entre telas distantes.
- Estado global (arquiteturas reativas): necessário para informações que permeiam toda a aplicação, como o status de autenticação, o carrinho de compras ou as preferências do usuário.

```
// Abordagem Local (setState): Risco de acoplamento e lógica na View

setState(() {
  _estaCarregando = true;
});

// Abordagem Profissional (Gerenciamento de Estado Centralizado - Ex: Provider/
Bloc)

context.read<AuthViewModel>().realizarLogin(usuario, senha);
```

- Provider: baseado na injeção de dependência, é a solução recomendada para iniciantes e projetos de médio porte por sua simplicidade.
- Riverpod: uma evolução do Provider, que oferece segurança em tempo de compilação e facilita testes automatizados sem depender da árvore de widgets.
- BLoC: utiliza fluxos de dados (streams) para separar completamente a lógica da interface. É o padrão ouro para aplicações de larga escala que exigem previsibilidade total de estados.

Quadro 19 – Comparativo técnico de gerenciamento de estado

Solução	Paradigma	Escalabilidade	Recomendação de ADS
setState	Imperativo/local	Baixa	Apenas para estados efêmeros de widget
Provider	Injeção de dependência	Média/alta	Projetos que buscam produtividade e baixo boilerplate
BLoC	Event-driven (Streams)	Altíssima	Sistemas complexos com múltiplas fontes de dados e testes rigorosos
Riverpod	Global reativo	Alta	Projetos modernos que exigem robustez e segurança de tipos

7.7 Garantia de qualidade: testes e estabilidade da aplicação

- Testes unitários: validam a menor unidade de código logicamente isolada (funções, métodos de modelos ou lógicas de cálculo). São rápidos e executados fora do ambiente do dispositivo.
- Testes de widget (UI Tests): verificam se os componentes da interface (botões, campos de texto, listas) são renderizados corretamente e se respondem adequadamente às interações básicas.
- Testes de integração: validam o fluxo completo da aplicação, desde a interação na interface até a persistência no banco de dados local ou a comunicação com o servidor remoto.

```
import 'package:flutter_test/flutter_test.dart';

import 'package:app_exemplo/models/validador.dart';

void main() {

  group('Validação de Autenticação', () {

    test('Deve retornar erro se a senha tiver menos de 6 caracteres', () {
```

Unidade IV

```
    final resultado = Validador.validarSenha('123');

    expect(resultado, 'Senha muito curta');

  });

  test('Deve retornar null (sucesso) para senhas válidas', () {

    final resultado = Validador.validarSenha('senha123');

    expect(resultado, isNull);

  });

}
```

Além dos testes funcionais, a engenharia de software mobile exige atenção a métricas não funcionais, como:

- Consumo de memória e bateria: testes que

monitoram se o gerenciamento de estado está causando memory leaks ou processamento excessivo. • Acessibilidade (accessibility): verificação de contraste, suporte a leitores de tela e tamanhos de fonte dinâmicos, garantindo a inclusão de todos os perfis de usuários.

Quadro 20 – Matriz de cobertura de testes

Tipo de teste	Alvo de validação	Ambiente de execução
Unitário	Lógica de negócio e models	Host (local)
Widget	Comportamento de componentes UI	Simulado (WidgetTester)
Integração	Fluxo CRUD e chamadas de API	Emulador/dispositivo real

8 SEGURANÇA, ENGENHARIA DE QUALIDADE E PUBLICAÇÃO

8.1 Autenticação de usuários e gestão segura de sessão

A autenticação é o processo de verificação da identidade de um usuário, enquanto a autorização define os privilégios de acesso a recursos específicos. Em arquiteturas mobile modernas, a autenticação baseada em tokens substitui o envio constante de credenciais, minimizando a exposição de dados sensíveis em trânsito. Para tanto, há protocolos de autenticação modernos, como JWT (JSON Web Token) e OAuth2, a serem utilizados.

- Arquitetura de fluxo com refresh token: para equilibrar segurança e experiência do usuário, utiliza-se o conceito de refresh tokens:
 - Handshake inicial: o usuário envia credenciais via HTTPS. – Emissão de par de tokens: o servidor retorna um access token (expira em minutos) e um refresh token (expira em dias/meses).
 - Autorização: o app utiliza o access token para requisições de API.
 - Renovação silenciosa: quando o access token expira, o app utiliza o refresh token para obter novas chaves sem interromper a navegação do usuário.
- Persistência segura (criptografia em repouso): dados sensíveis, como tokens de acesso e chaves de API, nunca devem ser armazenados em texto simples

(SharedPreferences comum). No desenvolvimento profissional, utiliza-se o hardware do dispositivo para proteção:

- Android KeyStore: protege chaves de criptografia.
- iOS Keychain: armazena segredos de forma isolada.

```
import 'package:flutter_secure_storage/flutter_secure_storage.dart';  
  
// Instancia o armazenamento criptografado (padrão OWASP)  
  
const storage = FlutterSecureStorage();
```

Unidade IV

```
// Escrita segura de token  
  
await storage.write(key: 'jwt_token', value: 'token_recebido_da_api');  
  
// Leitura protegida  
  
String? token = await storage.read(key: 'jwt_token');
```

- Diretrizes da OWASP Mobile Security: de acordo com o guia da OWASP Foundation (s.d.), a segurança deve ser implementada em profundidade: – Proteção da camada de rede: uso mandatório de HTTPS com validação de certificados (SSL Pinning) para evitar ataques Man-in-the-Middle. – Validação de entrada: sanitização rigorosa de dados antes do processamento. – Autorização granular: garantia de que o servidor valide se o usuário logado possui permissão específica para cada recurso solicitado (princípio do menor privilégio).

8.2 Engenharia de segurança e proteção de ativos digitais

Comunicação segura e SSL Pinning

Esta técnica consiste em chumbar a chave pública do certificado do servidor dentro do aplicativo, impedindo que atacantes interceptem o tráfego mesmo que possuam certificados fraudulentos instalados no dispositivo (ataques Man-in-the-Middle).

criptografia tanto em trânsito quanto em repouso: • Em trânsito: utilização de algoritmos de criptografia assimétrica para troca de chaves e simétrica para o fluxo de dados. • Em repouso: uso de áreas isoladas de hardware, como o secure enclave (iOS) e trusted execution environment (Android), para armazenar segredos e identificadores biométricos.

```
// Validação e Sanitização de Entrada no Nível de Aplicação

String sanitizarEntrada(String input) {

    // Remove caracteres que poderiam causar ataques de Scripting ou Injeção
    return input.replaceAll(RegExp(r' [<>{}\\[\]]'), '');

}

// Implementação de Timeout de Sessão

Timer(const Duration(minutes: 5), () {

    authService.logout(); // Encerramento preventivo por inatividade

});
```

Conjuntamente, o acesso a recursos nativos (câmera, GPS, contatos) deve seguir o princípio do menor privilégio: a aplicação deve solicitar apenas o estritamente necessário para sua operação. • Declaração: definir permissões nos arquivos de manifesto (AndroidManifest.xml e Info.plist). • Verificação em tempo de execução (runtime permissions): solicitar autorização no momento exato do uso, explicando o propósito ao usuário. • Graceful degradation: o sistema deve continuar operando, com funcionalidades reduzidas, caso o usuário negue o acesso.

Quadro 21 – Ameaças OWASP mobile aplicadas

Vulnerabilidade	Descrição técnica	Mitigação sugerida
M1: uso inadequado de plataforma	Falha no uso de TouchID ou Keychain	Seguir as diretrizes de API da Apple/Google
M2: armazenamento inseguro	Gravação de tokens em logs ou XML simples	Uso de flutter_secure_storage
M3: comunicação insegura	Tráfego via HTTP ou falta de validação de certificado	Implementar HTTPS e SSL Pinning
M4: autenticação insegura	Senhas fracas ou falta de refresh tokens	Implementar OAuth2 com JWT

8.3 Engenharia de qualidade: estratégias de testes automatizados

Uma estratégia de testes eficiente segue o conceito da pirâmide de testes, que prioriza a execução de testes mais rápidos e baratos na base, reservando os testes mais complexos para o topo.

- Testes unitários (base): validam a lógica pura de funções, métodos e modelos. São fundamentais para testar regras de negócio sem depender de interface ou rede.
- Testes de widget (meio): exclusivos do ecossistema Flutter, verificam se um componente visual (widget) renderiza corretamente e responde a eventos (como cliques) sem a necessidade de um emulador completo.
- Testes de integração (topo): validam o fluxo completo da aplicação, desde a interface até a resposta real de uma API ou banco de dados.

```
import 'package:flutter_test/flutter_test.dart';
import 'package:app_exemplo/screens/login_page.dart';

void main() {
  testWidgets('Deve validar se o botão de login está desabilitado sem
  credenciais',
    (WidgetTester tester) async {
      // Carrega o widget na memória para teste
      await tester.pumpWidget(const MaterialApp(home: LoginPage()));
```

20

DESENVOLVIMENTO MOBILE

```
      // Verifica se o texto 'Entrar' está presente
      expect(find.text('Entrar'), findsOneWidget);

      // Simula o clique e verifica se a validação foi disparada
      await tester.tap(find.byType(ElevatedButton));
      await tester.pump(); // Atualiza o estado do widget
      expect(find.text('Preencha os campos'), findsOneWidget);
    });
}
```

Métricas de desempenho: monitoramento de quadros por segundo (FPS) para garantir fluidez (60 FPS ou 120 FPS). • Acessibilidade: uso de ferramentas como o AccessibilityScanner para garantir que a aplicação seja utilizável por pessoas com deficiência. • Integração contínua (CI): configuração de pipelines (GitHub Actions ou GitLab CI) que executam todos os testes automaticamente a cada novo envio de código (push).

8.4 Testes multidispositivo, fragmentação e responsividade

O desenvolvimento para ecossistemas móveis apresenta o desafio da fragmentação, que ocorre devido à vasta diversidade de hardware, resoluções de tela e versões de sistemas operacionais, a compatibilidade deve ser garantida através de uma estratégia de testes que contemple não apenas a funcionalidade, mas a adaptação fluida da interface e do desempenho em diferentes cenários.

- Densidade de pixels (DPI): interfaces devem usar unidades independentes de densidade (dp/pt) para garantir que elementos mantenham o tamanho físico em telas com diferentes resoluções.
- Aspect ratio: a proporção da tela (por exemplo: 16:9, 18:9, 21:9) exige que o layout seja flexível para evitar cortes ou espaços vazios indesejados.
- Versões de API: o aplicativo deve declarar a `minSdkVersion` (compatibilidade mínima) e a `targetSdkVersion` (versão para a qual foi otimizado), tratando programaticamente as diferenças de recursos entre versões antigas e novas.
- MediaQuery: utilizado para obter as dimensões em tempo real e decidir a densidade de informações na tela.
- LayoutBuilder: permite que um widget conheça as restrições de espaço impostas por seu pai, ajustando-se internamente.

```
Widget build(BuildContext context) {
  var screenWidth = MediaQuery.of(context).size.width;

  return Scaffold(
    body: LayoutBuilder(
      builder: (context, constraints) {
        // Lógica de adaptação para Tablet vs Smartphone
        if (screenWidth > 600) {
          return const Row(children: [MenuLateral(), Conteudo()]);
        } else {
          return const Column(children: [Conteudo()]);
        }
      },
    ),
  );
}
```

- Emuladores e simuladores: ideais para testes rápidos de interface, verificação de fluxos lógicos e testes em versões específicas do SO sem custo de hardware.
- Dispositivos reais (physical devices): indispensáveis para validar o consumo de bateria, aquecimento, comportamento da câmera, sensores (GPS/Acelerômetro) e a fluidez real sob condições de rede instáveis.

Quadro 22 – Matriz de testes de compatibilidade

Fator de teste	Ferramenta/método	Objetivo de ADS
Resolução/layout	Device preview/emuladores	Garantir integridade visual em telas Notch e infinitas
Desempenho	Perfilamento (DevTools)	Identificar jank (travamentos) em aparelhos de entrada
Recursos nativos	Dispositivos físicos	Validar integração com biometria e sensores reais
Rede	Emulação de throttling	Testar resiliência em conexões 3G/Edge (Offline-first)

8.5 Ciclo de lançamento: otimização, build e versionamento

De acordo com as diretrizes da Google Play Store e Apple App Store, um aplicativo pronto para produção deve demonstrar eficiência no uso de recursos do sistema e transparência no gerenciamento de versões.

- Otimização de renderização: no Flutter, deve-se minimizar os rebuilds de widgets e utilizar o Skia/Impeller para garantir 60 FPS estáveis.
- Gestão de ativos (assets): imagens devem ser compactadas e fornecidas em múltiplas resoluções para evitar o

consumo excessivo de memória RAM. • Carregamento assíncrono e lazy loading: estratégias para carregar dados sob demanda, evitando o bloqueio da thread principal de interface.

O versionamento semântico (SemVer) é fundamental para o gerenciamento do ciclo de vida e manutenção. No desenvolvimento profissional, utiliza-se a estrutura MAJOR.MINOR.PATCH:

- VersionName (Ex: 1.2.0): a versão visível ao usuário e reflete mudanças significativas ou correções.
- VersionCode (Ex: 15): um número inteiro sequencial usado internamente pelas lojas para identificar qual build é mais recente.

Processo de build e assinatura digital

Diferente da versão de depuração (debug), a versão de produção (release) passa por um processo de otimização chamado tree shaking (remoção de código não utilizado) e deve ser assinada digitalmente.

Geração de Android App Bundle (AAB) - Formato otimizado para Google Play
`flutter build appbundle --release`

Geração de Build para iOS (Requer ambiente macOS e Xcode)
`flutter build ipa --release`

A assinatura digital garante a integridade e a autenticidade do software. No Android, utiliza-se uma chave privada (keystore); já no iOS, o processo envolve certificados de distribuição e perfis de provisionamento vinculados ao Apple Developer Program. Para evitar rejeições nas lojas, o desenvolvedor deve validar: • Políticas de privacidade: obrigatórias para qualquer app que manipule dados (conforme LGPD e GDPR). • Diretrizes de interface: o app deve seguir o Material Design (Android) e os Human Interface Guidelines (iOS) para garantir familiaridade ao usuário. • Declaração de permissões: justificar o uso de dados sensíveis (localização, contatos) nas descrições da loja.

Quadro 23 – Diferenças de publicação: Google Play versus App Store

Critério	Google Play Store (Android)	App Store (iOS)
Formato de entrega	.aab (App Bundle)	.ipa (ou via Xcode/Transporter)
Ferramenta de build	Android Studio/Gradle	Xcode/Fastlane
Tempo de revisão	Geralmente automático a poucas horas	Revisão manual humana (pode levar dias)
Custo de cadastro	Taxa única (One-time fee)	Anuidade (Annual subscription)

Após o lançamento, o desenvolvedor deve monitorar métricas de ANRs (App Not Responding) e crashes via console, utilizando esses dados para alimentar o backlog de correções e melhorias contínuas, mantendo a saúde do software em produção.

8.6 Engenharia de lançamento e ecossistemas de distribuição

A Google Play exige que o aplicativo seja submetido no formato Android App Bundle (.aab). Este formato utiliza o Dynamic Delivery, onde a loja gera APKs otimizados para a configuração específica do hardware do usuário (CPU, resolução e idioma), reduzindo o tamanho do download.

- Assinatura com Upload Key: uso do Gradle para assinar o binário com uma chave privada única.
- Configuração do console: definição de faixas de teste (Interno, Alpha, Beta) antes do lançamento em produção.
- Conformidade com a LGPD (Brasil, 2018): obrigatoriedade de informar como os dados são coletados, processados e excluídos (Data Safety Section).

O processo na Apple é mais rigoroso e exige o uso do Xcode em ambiente macOS. A publicação é baseada em um modelo de confiança estrito:

- Certificados de distribuição: vinculam o desenvolvedor à sua conta no Apple Developer Program.
- App IDs e Provisioning Profiles: arquivos que autorizam o aplicativo a utilizar recursos nativos (como Notificações Push ou iCloud) no hardware de produção.
- App Store Connect: plataforma de gerenciamento onde o binário (.ipa) é enviado via ferramenta Transporter ou Xcode.

Ao utilizar o Flutter, o desenvolvedor mantém uma única base de código, mas deve gerenciar dois processos de release distintos:

- Android: o comando flutter build appbundle prepara o código para o Google.
- iOS: o comando flutter build ipa gera o arquivo para a Apple, exigindo a abertura do projeto no Xcode para a assinatura final.

8.7 Síntese de engenharia: orquestração de uma aplicação completa

O fluxo técnico segue uma lógica de estados rigorosa:

- Inicialização e handshake de segurança: ao abrir o app, o sistema verifica a presença de um refresh token no secure storage. Se válido, o usuário é autenticado silenciosamente; caso contrário, é

direcionado à camada de view de autenticação. • Consumo de API e desserialização: após a entrada de dados, o service realiza uma chamada via HTTPS. A resposta (JSON) é convertida em um model de dados através de fábricas de mapeamento (factory constructors).

• Gerenciamento de estado reativo: o ViewModel ou Controller recebe o Model, processa a lógica de negócio e atualiza o estado global. A interface reage instantaneamente, reconstruindo apenas os widgets necessários (por exemplo, mudando o estado de um botão de “Carregando” para “Concluído”). • Navegação e contexto: com o sucesso da operação, o Navigator gerencia a transição para a tela principal, injetando o contexto do usuário autenticado para as próximas interações.

Quadro 25 – Matriz de integração de componentes

Componente	Função técnica	Tecnologia sugerida
Arquitetura	Separação de responsabilidades (SoC)	Clean Architecture/MVVM
Segurança	Proteção de credenciais e tráfego	JWT + SSL Pinning + Keychain
Estado	Sincronização entre lógica e UI	BLoC/Riverpod/Provider
Persistência	Armazenamento de dados em repouso	SQLite/Hive/Secure Storage
Qualidade	Validação da estabilidade do fluxo	Testes de integração e UI

Flutter tutorial

https://www.youtube.com/playlist?list=PL5EmR7zuTn_Yu_YV2pT0h0843vRGiTMtx